



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Lucas Pereira da Silva

Refatoração Automática de Código de Teste

Florianópolis
7 de junho de 2026

SUMÁRIO

1	INTRODUÇÃO	2
	REFERÊNCIAS	4

1 INTRODUÇÃO

Teste de software é uma das atividades presentes no processo de desenvolvimento e manutenção de software. Conforme o *Guide to the Software Engineering Body of Knowledge* (SWEBOK) (Bourque; Fairley et al., 2014) define, teste de software consiste na verificação dinâmica de que um programa provê um comportamento esperado a partir de um conjunto finito de casos de teste, adequadamente selecionados a partir de um domínio de execução potencialmente infinito. Por muito tempo, teste de software foi visto sobretudo como uma etapa de verificação da qualidade do software produzido, e não como meio de construir um produto com qualidade (Murugesan, 1994). Nesse sentido, testes eram frequentemente realizados apenas ao final da etapa de codificação (Yang et al., 2011), com o objetivo de detectar falhas e fornecer um indicativo de que o software funciona conforme o esperado (Bertolino, 2007). Com o passar do tempo, a percepção sobre a utilidade do teste evoluiu. A atividade deixou de ser vista como etapa posterior ao desenvolvimento e passou a estar presente durante todo o processo de desenvolvimento e manutenção de software (Bourque; Fairley et al., 2014). Meszaros (2007) considera a atividade de teste parte intrínseca do processo de desenvolvimento de software. Assim, a atividade de teste de software passou a incorporar propósitos variados, como detectar falhas (Tiwari; Goel, 2013), prevenir que erros sejam introduzidos (Beizer, 1990), prover um indicativo de que o software funciona (Bertolino, 2007), auxiliar na compreensão do projeto e do código do sistema (Freeman; Pryce, 2009) e servir como meio para especificação (Alvestad, 2007) e documentação (Haugset; Hanssen, 2008) dos requisitos.

Nesse contexto, o código de teste deixou de ser um artefato auxiliar e passou a exigir os mesmos cuidados aplicados ao código de produção. Assim como o código da aplicação, o código de teste precisa ser mantido, compreendido e ajustado durante todo o projeto (Greiler; Deursen; Storey, 2013b). Para Meszaros (2007), a atividade de teste não deveria aumentar o esforço total de desenvolvimento e manutenção do software, uma vez que o esforço gasto com essa atividade deve ser compensado pela melhoria da qualidade do software. Entretanto, manter o código de teste apresenta desafios, principalmente quando a codificação e a evolução dos testes são realizadas diretamente pelo desenvolvedor. Conforme o software evolui e a quantidade de código aumenta, torna-se mais difícil manter os testes com consistência e sem perder o controle sobre eles. Fenômeno semelhante ocorre no código de produção: sem refatoração contínua, cada nova funcionalidade tende a exigir mais esforço do que a anterior (Fowler, 1999). Com os testes, observa-se comportamento análogo. Novos testes passam a ser mais difíceis de adicionar e a duplicação tende a aumentar (Berner; Weber; Keller, 2005). Isso pode levar à degradação da manutenibilidade do código de teste (Greiler; Deursen; Storey, 2013a). Segundo Emery (2009), um motivo comum para o abandono de testes automatizados é o fato de os testes se tornarem frágeis e de alto custo de manutenção, o que pode tornar os desenvolvedores reticentes tanto em adicionar novos testes quanto em executar os existentes.

Evitar a duplicação no código de teste é uma das formas de melhorar a manutenibilidade. Isso porque mudanças na aplicação passam a ter impacto menor nos testes, e o desenvol-

vedor dispõe de menos código para manter e gerenciar (Berner; Weber; Keller, 2005; Greiler; Deursen; Storey, 2013a). De acordo com Berner, Weber e Keller (2005), testes tendem a ser repetitivos e apresentam alto potencial de reuso; conforme o software evolui e o número de testes aumenta, é comum que a quantidade de código duplicado entre os testes também aumente. Diante disso, técnicas que reduzam essa duplicação podem contribuir para preservar a manutenibilidade do código de teste ao longo do projeto.

REFERÊNCIAS

- ALVESTAD, K. **Domain Specific Languages for Executable Specifications**. Dissertação (Mestrado) — Institutt for datateknikk og informasjonsvitenskap, 2007.
- BEIZER, B. **Software Testing Techniques**. 2. ed. [S.l.]: Van Nostrand Reinhold Company, 1990.
- BERNER, S.; WEBER, R.; KELLER, R. K. Observations and lessons learned from automated testing. In: **Proceedings of the 27th International Conference on Software Engineering**. [S.l.]: ACM, 2005. p. 571–579.
- BERTOLINO, A. Software testing research: Achievements, challenges, dreams. In: **2007 Future of Software Engineering**. [S.l.]: IEEE Computer Society, 2007. p. 85–103.
- BOURQUE, P.; FAIRLEY, R. E. et al. **Guide to the software engineering body of knowledge (SWEBOK): Version 3.0**. [S.l.]: IEEE Computer Society Press, 2014.
- EMERY, D. H. Writing maintainable automated acceptance tests. In: **Agile Testing Workshop, Agile Development Practices**. Orlando, Florida: [s.n.], 2009.
- FOWLER, M. **Refactoring: Improving the Design of Existing Code**. [S.l.]: Addison-Wesley, 1999.
- FREEMAN, S.; PRYCE, N. **Growing Object-Oriented Software, Guided by Tests**. [S.l.]: Pearson Education, 2009.
- GREILER, M.; DEURSEN, A. van; STOREY, M.-A. Automated detection of test fixture strategies and smells. In: **2013 IEEE Sixth International Conference on Software Testing, Verification and Validation**. [S.l.]: IEEE, 2013. p. 322–331.
- GREILER, M.; DEURSEN, A. van; STOREY, M.-A. Strategies for avoiding text fixture smells during software evolution. In: **Proceedings of the 10th Working Conference on Mining Software Repositories**. [S.l.]: IEEE Press, 2013. p. 387–396.
- HAUGSET, B.; HANSSEN, G. K. Automated acceptance testing: A literature review and an industrial case study. In: **Agile 2008 Conference**. [S.l.]: IEEE, 2008. p. 27–38.
- MESZAROS, G. **xUnit test patterns: Refactoring test code**. [S.l.]: Pearson Education, 2007.
- MURUGESAN, S. Attitude towards testing: a key contributor to software quality. In: **Proceedings of 1994 1st International Conference on Software Testing, Reliability and Quality Assurance (STRQA'94)**. [S.l.: s.n.], 1994. p. 111–115.
- TIWARI, R.; GOEL, N. Reuse: reducing test effort. **ACM SIGSOFT Software Engineering Notes**, v. 38, n. 2, p. 1–11, 2013.
- YANG, Y. et al. Testqual: Conceptualizing software testing as a service. **e-Service Journal**, Indiana University Press, v. 7, n. 2, p. 46–65, 2011. ISSN 15288226, 15288234. Disponível em: <http://www.jstor.org/stable/10.2979/eservicej.7.2.46>.